# Check your status then complete

I was cut off mid-sentence in Lane 1 — `DockerCompose` description. Completing from that exact point through all remaining lanes, the full AEU execution plan, and the closing section now.

*(Continuing from Lane 1 — Foundation)*

`DockerCompose`
*Every function:* `docker-compose.yml` (dev — all 6 services with volume mounts and hot reload) → `docker-compose.prod.yml` (production — resource limits, restart policies, no volume mounts) → `Dockerfile` per service (API, Celery worker, Celery Beat, Ollama sidecar) → service startup order enforcement (Postgres → Redis → Ollama → Celery Beat → Celery Worker → API) → health check script per service → `docker compose up --dry-run` self-test → network and volume definitions → secret injection from environment (no hardcoded values anywhere).
*Gate:* All 6 services start clean. Health checks pass. API reachable at `localhost:8000/health`.

`EnvConfig`
*Every function:* `.env.example` with every required variable documented inline → Pydantic `BaseSettings` `config.py` with type validation on all vars → `validate_env.sh` (fails with explicit error message on any missing required var before any service starts) → secret reference map (which vars come from GCP Secret Manager in production vs `.env` in dev) → Stripe key configuration → GCS bucket configuration → Ollama endpoint configuration → self-test: run `validate_env.sh` against `.env.example` with all required vars present — must pass. Run with one required var removed — must fail with clear error.
*Gate:* Validation script passes on complete env. Fails loudly on incomplete env.

`CICDBuilder`
*Every function:* `ci.yml` (triggers on every PR — runs LinterFixer, full test suite, eval gate check) → `deploy-canary.yml` (builds Docker image, pushes to Artifact Registry, deploys new Cloud Run revision at 10% traffic, sets 48h observation hold) → `deploy-promote.yml` (promotes canary revision to 100% traffic after gate criteria met) → `rollback.yml` (one-command rollback to previous revision — completes in under 60 seconds) → `eval-gate.yml` (runs golden set evaluation, blocks promotion if any metric below threshold) → GitHub branch protection rules (no direct push to `main`, PR requires CI pass + CriticAgent approval label) → secrets injection from GitHub Secrets → `act` local runner self-test.
*Gate:* All 5 workflow files valid YAML. Branch protection configured. Test PR triggers CI correctly.

**Lane 1 Gate:** All migrations run clean on fresh Postgres. All 6 Docker services start. All env vars documented and validated. CI workflow fires on test PR. Schema contract published to `AGENTS.md`.

## Lane 2 — Backend Core (5 agents, fully parallel)

*All 5 agents run in parallel. All reference the finalized schema contract from Lane 1's* `AGENTS.md`.

`APIBuilder`
*Every function:* All FastAPI 0.115 routes with full Pydantic v2 request/response models → dependency injection setup (DB session, tenant context, auth) → background task registration (billing event write is async — never blocks query response) → global error handler (maps all custom exceptions to correct HTTP status codes with structured JSON error body) → OpenAPI tag grouping (auth, search, documents, citations, monitoring, admin) → middleware registration order (PrivacyGuard → AuthGuard → RateLimiter → AuditLogger → TenantContextSetter → CORSMiddleware) → route-level integration tests (httpx.AsyncClient against test DB) → self-test: all routes return correct status codes on valid and invalid payloads.

`MCPServerAgent`
*Every function:* Full implementation of all 8 MCP tools from the contracts defined in Part III → MCP SDK protocol compliance (correct message types, correct error formats) → capability manifest builder (reads live from `jurisdictions` table — always current) → streaming response setup for `research_task` (WebSocket stream so clients see results as they arrive, not all at once) → tool input validation (Pydantic on all tool arguments) → MCP inspector self-test run → tool schema validation test (all 8 tools pass MCP schema validation).

`AuthGuard`
*Every function:* API key hash-verify function (bcrypt hash stored in `tenants.api_key_hash`, plaintext never stored) → JWT encode/decode (access token 30min, refresh token 7 days) → rate limiter middleware (per-tenant, per-tier: Solo=100 req/min, Firm=1000 req/min, Enterprise=unlimited, Developer=per-API-key config) → RLS context setter (`SET LOCAL app.tenant_id = $1` on every DB connection) → tier-based feature gate (conflict_check tool only available Firm+ tier) → audit log write on every authenticated request (fingerprint only — no raw query) → self-test: unauthenticated request returns 401, wrong key returns 403, rate limit exceeded returns 429.

`PrivacyGuard`
*Every function:* PII pattern scanner (regex patterns for Canadian SIN, US SSN, email addresses, phone numbers, health card numbers, names in legal-context patterns) → request rejection on PII detection (400 with `PRIVACY_VIOLATION` error code, structured response including which pattern was matched) → `X-LexCore-Privacy-Policy` header injection on first connection per tenant → raw query content interception (confirms query text never touches any logging sink — enforced as middleware, not configuration) → PIPEDA disclosure document endpoint (`GET /privacy`) → self-test: send query containing a SIN pattern — must be rejected. Send clean query — must pass.

`CacheLayer`
*Every function:* Redis 7 client setup with connection pooling → query fingerprint function (SHA256 of normalized query + all filter parameters, sorted deterministically — identical inputs always produce identical fingerprint) → cache get/set middleware → TTL configuration table (search_legal=3600s, research_task=1800s, get_document=21600s, get_citations=21600s,

check_updates=900s, jurisdiction_summary=3600s, get_capabilities=3600s) → cache invalidation triggers (on `content_hash` change in `legal_documents` — invalidates all cache entries referencing affected document IDs) → `hit_count` increment on cache hit → cache metrics (hit rate per tool) → self-test: identical query twice — second must show `cache_hit: true`. Document update — subsequent query must bypass cache and return fresh result.

**Lane 2 Gate:** All routes return correct status codes. MCP tools pass schema validation. PII in query payload rejected. Unauthenticated requests rejected. Cache hit/miss correctly logged. RLS confirmed — tenant A cannot see tenant B's audit log.

## Lane 3 — Data Pipeline (8 agents, IngestArchitect first then 7 parallel)

`IngestArchitect` *(runs first, publishes task contracts before others start)*
*Every function:* Complete Celery 5 task architecture (task signatures, queue routing, worker concurrency settings) → Celery Beat schedule (all Phase 1 sources with correct cadences from Section 4.1) → retry policy configuration (3 attempts, exponential backoff: 5m→25m→125m) → dead-letter queue routing (after 3 retries → `ingest_jobs.status = 'dead_letter'` + SRE alert) → Flower monitoring setup (Celery task dashboard at `/flower`) → `ingest_jobs` table write contract published to `AGENTS.md` → SourceAdapter base class finalized → self-test: queue one test task, verify retry behavior on simulated failure.

*After IngestArchitect gate pass — 6 agents run in parallel:*

`SourceAdapter[CanLII]`
*Every function:* CanLII API connection (rate-limited per CanLII ToS post-partnership) → bilingual document fetch (EN + FR parallel) → `title_fr` extraction → license validation check (reads `LEGAL_DATA_SOURCES.md` — refuses if CLO clearance not confirmed) → format change detection (structure hash comparison against last known good) → raw archive to GCS (`northamerica-northeast1` bucket) → `legal_documents` insert/update → chunk task publish to Celery → `ingest_jobs` log write per batch.

`SourceAdapter[eCFR]`
*Every function:* eCFR XML API consumption (no scraping — structured API) → regulation hierarchy extraction (Part→Section→Subsection → `section_path`) → daily diff detection (eCFR provides amendment notifications) → US Public Domain license auto-cleared → full text extraction → GCS archive → `legal_documents` write → chunk task publish.

`SourceAdapter[JusticeLaws]`
*Every function:* Justice Laws Canada HTML scraper (Playwright) → bilingual HTML parsing (EN + FR column extraction) → `title_fr` population → Crown Copyright license check (CLO clearance required before writing) → document structure extraction → OCR quality scoring on PDF-sourced documents → GCS archive → `legal_documents` write → chunk task publish.

`SourceAdapter[CourtListener]`
*Every function:* CourtListener REST API consumption (CC-BY 4.0 — auto-cleared) → attribution metadata field population (`license_type = 'cc_by'`, source URL preserved) → case law structure extraction (court, opinion, cited cases) → initial citation pair extraction (passes

to `CitationGraphAgent` via task queue) → federal + state court coverage (Phase 1: federal only) → GCS archive → write → chunk task publish.

`SourceAdapter[StateStatutes]`
*Every function:* Playwright-based scraper (per-state format adapters) → Phase 1 states only: US-CA, US-NY, US-TX, US-FL, US-WA → per-state ToS validation (reads clearance from `LEGAL_DATA_SOURCES.md`) → format change sentinel (compares page structure hash) → if format changes: auto-pause that state's ingest, write `INC-001` trigger to alert queue → GCS archive → write → chunk task publish.

`ChunkingEngineer`
*Every function:* Celery `chunk_document` queue consumer → document structure parser (detects Act/Regulation/Case structure from `doc_type`) → hierarchy extractor (Part/Division/Section/Subsection/Paragraph) → `section_path` builder (human-readable breadcrumb string) → token counter (tiktoken) → 512-token max chunk size enforcement → 50-token overlap between adjacent chunks → quality score per chunk (OCR confidence from Docling output) → reject chunk if `quality_score < 0.7` → write `legal_chunks` rows (without embedding yet) → publish embed tasks to Celery `embed_chunk` queue → bilingual parallel chunk creation.

`EmbeddingWorker`
*Every function:* Celery `embed_chunk` queue consumer → Ollama client (`mxbai-embed-large-v1`, 1024d) → batch construction (32 chunks per Ollama call — optimal throughput) → embedding call with retry (3 attempts, Ollama restart on repeated timeout) → vector write to `legal_chunks.embedding` → `quality_score` update (combines chunking quality + embedding confidence) → batch metrics logging (chunks/minute, failures/batch) → dead-letter on repeated embedding failure → model version tag on all embeddings (critical for future re-embed jobs on model upgrade).

**Lane 3 Gate:** 1,000+ documents ingested per Phase 1 source. All chunks have embeddings. Dead-letter rate < 2% of total. `ingest_jobs` shows complete run logs with no silent failures. GCS archive verified for all ingested documents.

## Lane 4 — Search & Intelligence (3 agents, staged within lane)

`HybridSearchEngineer` *(runs first)*
*Every function:* pgvector cosine similarity query builder → PostgreSQL full-text search (`ts_rank` with `plainto_tsquery`) → hybrid score formula: `(semantic_weight × cosine_similarity) + (keyword_weight × ts_rank)` → configurable weight injection (from `search_config` parameter — never hardcoded) → multi-language query support (EN: `'english'` dictionary, FR: `'french'` dictionary, bilingual: union of both) → jurisdiction + body-of-law + doc-type filter injection → quality gate (`quality_score >= 0.7` filter on all chunks) → `data_freshness` metadata attachment per jurisdiction → result normalization (scores bounded 0.0–1.0) → pagination → performance benchmark self-test: top-10 search on 1M chunks must complete in < 200ms.

*After HybridSearchEngineer gate pass:*

`ResearchTaskAgent` *(runs second, integrates HybridSearch)*
*Every function:* Legal question decomposer (breaks complex question into 3–5 sub-queries using a structured prompt — e.g., "What are the notice requirements for terminating a contract in Ontario vs California?" decomposes into: (1) Ontario contract termination notice, (2) California contract termination notice, (3) cross-border enforcement considerations) → parallel asyncio search across all sub-queries (using `HybridSearchEngineer`'s query builder) → result deduplication (same chunk_id from multiple sub-queries → keep highest relevance score) → jurisdiction gap detection (which requested jurisdictions returned zero results) → top-source ranker → structured research report assembly → confidence scorer (weighted average of top result scores) → data freshness metadata per jurisdiction → DISCLAIMER field injection → `source_chunk_ids[]` array on all outputs (full claim traceability) → self-test: 10 complex questions produce structured reports with all required fields.

*After ResearchTaskAgent gate pass:*

`CitationGraphAgent` *(runs third)*
*Every function:* Citation pair extractor from case law text (structured field extraction first, regex fallback for inline citations) → citation type classifier (`references`, `overrules`, `amends`, `repeals`, `implements`, `upholds`, `distinguishes`) → confidence score on regex-extracted citations (< 1.0 to distinguish from structured field extractions) → `legal_citations` bulk writer → BFS graph traversal (up to depth 3, respects direction parameter: upstream/downstream/both) → chain-of-authority builder (ordered list from most authoritative to most recent) → overruled case detector (if case A `overrules` case B, set `is_current = FALSE` on case B's document and write `legal_updates` record) → document flag updater → self-test: 3 known precedent chains from CourtListener produce correct authority ordering.

**Lane 4 Gate:** Precision@5 ≥ 0.80 on 20-question pre-golden sample. `research_task` returns complete structured reports with all required fields on 5 complex questions. 3 known citation chains produce correct authority order. Overruled case detection confirmed on 2 known examples.

## Lane 5 — Quality (3 agents, fully parallel)

`TestWriter`
*Every function:* Unit tests for every pure function (pytest + pytest-asyncio) → integration tests for every API route (httpx.AsyncClient against isolated test DB with transaction rollback per test) → fixture factory (tenant fixtures, document fixtures, chunk fixtures, all jurisdiction seeds) → golden set harness (`tests/golden_set.json` loader, 50 cases, structured query format) → mock Ollama client (deterministic embedding vectors for test repeatability) → mock GCS client → mock Stripe client → mock source adapter (deterministic document list) → coverage report configuration (`pytest --cov=app --cov-report=term-missing`) → self-test: `pytest tests/ -v --cov=app` must show > 85% coverage. All tests pass on fresh environment.

`EvalRunner`
*Every function:* Golden set loader and runner → `search_legal` caller for each test case → result scorer (Precision@5: are top 5 results in expected set? Recall@10: are all expected

results in top 10? NDCG@10: is ranking quality correct?) → LLM-as-judge execution (structured prompt against human annotations, score 0–5) → judge calibration check (score delta from human < 0.5 on all calibration cases) → `citation_hallucination_rate` check (every `LexDraft/LexAnalysis` output — every claim must have valid `source_chunk_id`) → `data_freshness` check (all docs within expected sync window) → baseline comparison (block if any metric drops > 5%) → `eval_runs` table write (all metrics + git SHA + gate pass/fail) → self-test: deterministic test input produces deterministic score across 3 runs.

`LinterFixer`
*Every function:* `black .` (auto-format all Python files) → `isort .` (auto-sort all imports) → `flake8 app/ tests/` (report remaining style violations as structured JSON) → `mypy app/ --strict` (full type checking — no implicit Any, no missing type annotations) → auto-fix commit for black + isort changes → remaining `flake8` + `mypy` errors reported to `CriticAgent` as structured JSON → **gate fails if any** `mypy --strict` **errors remain** — type safety is non-negotiable in a legal context → self-test: `mypy app/ --strict` exits 0.

**Lane 5 Gate:** > 85% test coverage. All `mypy --strict` checks pass. Eval harness runs to completion on seed data. Golden set infrastructure ready (populated by legal SME — async human step that can run during AEU-5 without blocking AEU-6).

## Lane 6 — Ops, Integrations & Docs (4 agents, fully parallel)

`ObservabilityAgent`
*Every function:* Prometheus counter/histogram definitions (request_total, request_duration_seconds, ingest_documents_total, ingest_failures_total, cache_hit_total, eval_precision_at_5, query_limit_utilization_pct) → metric decorator injection on all FastAPI routes and all Celery tasks → structured JSON logging format (every log line: timestamp, level, correlation_id, tenant_id_hash, agent_id, tool_called, latency_ms, cache_hit — NO raw query content) → correlation ID middleware (UUID injected on every request, propagated through all service calls) → Loki log aggregation configuration → Grafana dashboard JSON (4 panels: request rate by tool, error rate, P95 latency by tool, ingest jobs/hour) → alert rules YAML (error_rate > 0.5%, P95 > 400ms for search_legal, P95 > 3s for research_task, ingest_lag > 2× expected cadence, dead_letter_depth > 50) → PagerDuty routing for critical alerts → self-test: generate 100 test requests, verify all metrics appear correctly in Prometheus, all logs correctly structured.

`SREAgent`
*Every function:* `/health` liveness endpoint (fast — just returns 200 if process alive) → `/ready` readiness endpoint (checks: Postgres connection, Redis ping, Ollama model loaded, GCS bucket accessible — returns 503 with structured failure detail if any dependency unavailable) → `/metrics` Prometheus endpoint → five incident runbooks (`incidents/INC-001.md` through `INC-005.md` — each with: detection criteria, immediate actions, investigation steps, resolution steps, post-incident review template) → backup script (`scripts/backup.sh` — pg_dump + GCS upload with timestamp, 30-day retention) → restore script (`scripts/restore.sh` — restore from GCS backup, verify row counts post-restore) → `oncall.md` (escalation path: SRE alert → engineer → CTO → CEO for P0 incidents) → self-test: kill Postgres container, verify `/ready` returns 503 with Postgres identified as failing dependency.

`IntegrationAdapter` *(new agent added from Gap #4)*
*Every function:* `lexconnect-clio` adapter (Clio API OAuth2 setup, matter context reader, research result writer as matter note, Clio App Marketplace manifest) → `lexconnect-netdocuments` adapter (ndConnect program API, matter workspace reader, research memo writer to document library, NetDocuments authentication) → `lexconnect-imanage` adapter (iManage Work 10 REST API, matter folder reader, research document writer, iManage authentication) → shared `LexConnectBase` class (standardizes MCP client → LexCore gateway → result formatting pipeline across all three adapters) → integration test suite (mock each DMS API, verify correct matter context extraction and result delivery) → self-test: mock Clio matter with known practice area → verify correct jurisdiction + body_of_law passed to `LexRouter`.

`DocWriter`
*Every function:* `README.md` (quick start in < 5 commands, architecture overview, configuration reference, troubleshooting) → `ARCHITECTURE.md` (full system design + 7 Architecture Decision Records: pgvector choice, Celery choice, mxbai model choice, RLS pattern, MCP gateway pattern, GCP northamerica-northeast1 region choice, Stripe Meters API choice) → `API.md` (all 8 MCP tools with full examples, all REST endpoints with request/response examples, error code reference) → `AGENTS.md` template (the living project manifest — initialized with all lane contracts, ready for ContextKeeper to populate) → `LEGAL_DATA_SOURCES.md` (complete license clearance registry — all sources, license type, CLO clearance status, required action, clearance date when approved) → `LEGAL_DISCLAIMER.md` (full disclaimer text, embedding instructions for all product surfaces, API disclaimer field specification) → `GROUNDING_CONTRACT.md` (citation grounding contract for LexDraft + LexAnalysis — output schema, chunk_id requirement, enforcement specification) → `TENANT_ISOLATION_POLICY.md` (client-facing document for law society audits — explains cross-tenant prohibition, RLS architecture, audit log privacy design) → `PIPEDA_COMPLIANCE.md` (PIPEDA adequacy framework, Law 25 PIA summary for Quebec firms, data residency architecture, DPA template) → `INTEGRATIONS.md` (LexConnect setup for Clio, NetDocuments, iManage) → self-test: `README.md` quick start instructions executed on clean machine — system starts successfully.

**Lane 6 Gate:** Metrics appear in Prometheus on test traffic. Health endpoints return correct status on healthy system and correct 503 on dependency failure. All runbooks reviewed by CriticAgent for completeness. All 9 documentation files complete. `README.md` quick start verified working.

## AEU Execution Plan — Complete Gate Table

| AEU | Lane | Agents | Entry Criteria | Gate Criteria | Blocking Dependency |
|---|---|---|---|---|---|
| AEU-0 | Command & Control | HordeMaster, ContextKeeper, CriticAgent | Approved LexCore v4.0 spec (this document) | `AGENTS.md` initialized, all 7 lane contracts defined, CLO clearance confirmed for Phase 1 sources | CLO must clear CanLII + Justice Laws + CourtListener before AEU-1 |

| AEU | Lane | Agents | Entry Criteria | Gate Criteria | Blocking Dependency |
|---|---|---|---|---|---|
| AEU-1 | Foundation | SchemaArchitect, DockerCompose, EnvConfig, CICDBuilder (parallel) | AEU-0 gate passed | Migrations run clean, Docker 6-service start, env validation passes, CI fires on test PR, schema contract in `AGENTS.md` | Schema contract must be in `AGENTS.md` before any Lane 2 agent starts |
| AEU-2 | Backend Core | APIBuilder, MCPServerAgent, AuthGuard, PrivacyGuard, CacheLayer (parallel) | AEU-1 gate passed | All routes correct, MCP schema valid, PII rejected, auth rejects unauthorized, cache logs correctly, RLS isolation confirmed | MCP tool contracts published to `AGENTS.md` before Lane 3 |
| AEU-3 | Data Pipeline | IngestArchitect first → SourceAdapters×5 + ChunkingEngineer + EmbeddingWorker (parallel) | AEU-2 gate passed + Ollama running with mxbai model loaded | 1K+ docs per Phase 1 source, < 2% dead-letter, all chunks embedded, GCS archive verified, ingest jobs log complete | IngestArchitect task contracts published before SourceAdapters start |
| AEU-4 | Search & Intel | HybridSearch → ResearchTask → CitationGraph (staged within lane) | AEU-3 gate passed (10K+ documents with embeddings) | Precision@5 ≥ 0.80 on 20-question sample, research_task returns complete reports, 3 citation chains verified, overruled detection confirmed | HybridSearch must complete before ResearchTask integrates |
| AEU-5 | Quality | TestWriter, EvalRunner, LinterFixer (parallel) | AEU-4 gate passed | > 85% test coverage, all mypy --strict pass, eval harness operational, citation_hallucination_rate = 0.000 on seed outputs | Legal SME golden set annotation (async human step — can run during AEU-5/6 without blocking AEU-7) |
| AEU-6 | Ops, Integrations, Docs | ObservabilityAgent, SREAgent, IntegrationAdapter, DocWriter (parallel) | AEU-5 gate passed | Metrics in Prometheus, health endpoints correct, LexConnect adapters pass mock DMS tests, all 9 docs complete | None — runs concurrent with legal SME annotation |
| AEU-7 | Integration Gate | HordeMaster orchestrates all lanes | AEU-6 gate passed + legal SME annotations complete | All tests green, golden set Precision@5 ≥ 0.85, citation_hallucination_rate = 0.000, 48h canary stable, all eval metrics above gate thresholds | All prior gates must pass. No exceptions. |

# PART X — PHASE ROLLOUT TARGETS

Measured by data coverage and quality gates — not human time.

**Phase 1 — Canadian Sovereignty Wedge:**
CanLII (CA-FED + ON + BC + AB) + Justice Laws Canada bilingual → ~140,000 documents. All eval gates passing. 10 beta users onboarded. LexMonitor live. LexConnect-Clio adapter in Clio App Marketplace review. PIPEDA DPA template available for all tenants. Law 25 PIA complete.

**Phase 2 — North American Cross-Border:**

- eCFR + US Code + CourtListener federal + US-CA/NY/TX/FL/WA state statutes → ~800,000 documents. Cross-border US+CA research capability live — the differentiator Harvey cannot match without restructuring. 50+ paying customers. LexConnect-NetDocuments live. Developer API tier open.

**Phase 3 — Full Continental Coverage:**

- All 50 US states + all remaining CA provinces/territories + Quebec bilingual (Law 25 PIA per-firm available) → 3M+ documents. Full North American corpus. 200+ customers. Enterprise self-host tier live. pgvector performance reviewed against 50M vector threshold — Qdrant evaluated for potential addition.

# DOCUMENT SUMMARY

**What this is:** The complete, final, approved specification for LexCore v4.0 — North America's Legal Intelligence Database — incorporating all 8 team perspectives (Executive, Strategy, ICP, Engineering, Dev, Testing, Rollout, Maintenance), Karpathy's agentic engineering principles, the 26-agent Agentic Build Horde, and all 6 critical gaps fully resolved.[1] [2] [3] [4] [5] [6]

**What comes next:** AEU-0 code package — `AGENTS.md` manifest, `LEGAL_DATA_SOURCES.md` CLO checklist, and the complete Alembic migration set — ready to hand directly to your coding agents on your green light.[7]

**Single required human action before AEU-0:** CLO clearance sign-off on Phase 1 data sources (CanLII API partnership confirmation, Justice Laws Canada reproduction scope confirmation). Everything else executes autonomously.

**Green light?**

⁂


1. https://www.the-ai-corner.com/p/andrej-karpathy-ai-workflow-shift-agentic-era-2026
2. https://www.linkedin.com/pulse/agentic-shift-rewriting-organization-bassel-haidar-zcmze
3. https://news.agentcommunity.org/issues/2026-02-11-sovereign-swarms-and
4. https://aiautomationglobal.com/blog/agentic-coding-revolution-multi-agent-teams-2026
5. https://www.opsintel.io/blog/ai-compliance-for-canadian-law-firms-2026/

6. https://starterpick.com/blog/stripe-metered-billing-saas-boilerplate-2026
7. https://stackconsulting.pro